

Day 1 — Python Core I

Data Types & Control Flow — Notes for Recap

1. Python — Introduction

What is Python?

Python is a high-level, general-purpose programming language known for its simple, readable syntax (close to plain English). It is widely used in web development, automation, AI/ML, and — most relevant here — **Data Engineering**.

Why Python is used in Data Engineering:

- **Readable & fast to write** — pipelines are quicker to build, test, and debug
- **Rich ecosystem of libraries** — Pandas (data cleaning), PySpark (big data processing), Airflow (orchestration) are all Python-based
- **Acts as a "glue" language** — connects databases, APIs, cloud services, and file systems together in one workflow

Interpreted language: Python code runs line by line (unlike compiled languages like C or Java), which makes testing and debugging faster and more direct.

***Key takeaway:** Python is the connective layer that ties together every other tool in the data stack — which is why it's the first thing to learn in data engineering.*

2. Variables & Data Types

What is a variable?

A variable is a named container used to store a value in memory, so it can be reused later in the program.

```
name = "Akash"    # stores the text "Akash" in the variable 'name'
age = 21          # stores the number 21 in the variable 'age'
```

Note: Python does not require you to declare a type (like `int` or `string`) before assigning a variable — the type is inferred automatically. This is called **dynamic typing**.

Core Data Types

Type	Description	Example
int	Whole number	age = 21
float	Decimal number	price = 99.5
str	Text (string)	name = "Akash"
bool	Boolean value (True/False)	is_active = True

```
print(type(age))    # <class 'int'>
print(type(price))  # <class 'float'>
```

Type Conversion (Type Casting)

Converting a value from one data type to another.

```
x = "25"
y = int(x)    # string "25" -> integer 25
z = str(100)  # integer 100 -> string "100"
```

Why this matters in Data Engineering: Data coming from CSV files or APIs is almost always read as **text (str)** by default — even if it looks like a number. Before performing any calculation, that value must be explicitly converted to `int` or `float`. Forgetting this is one of the most common bugs when building data pipelines.

***Key takeaway:** A variable is a named container; a data type defines the kind of value it holds. Type conversion is essential when handling real-world, file-based, or API data.*

3. Lists & Tuples

What is a List?

A list is an ordered, **mutable** (changeable) collection of items. You can add, remove, or modify elements after creation.

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")    # ['apple', 'banana', 'cherry', 'mango']
fruits[0] = "orange"      # ['orange', 'banana', 'cherry', 'mango']
```

What is a Tuple?

A tuple is an ordered, **immutable** (unchangeable) collection of items. Once created, its values cannot be modified.

```
coordinates = (10, 20)
# coordinates[0] = 99    # Error - tuples cannot be modified
```

Feature	List	Tuple
Mutable?	Yes	No
Syntax	[]	()
Use case	Data that changes (cart items, logs)	Data that must stay fixed (coordinates, config)

***Key takeaway:** Use a list when the data will change over time. Use a tuple when the data should remain constant and protected from accidental modification.*

4. Sets & Dictionaries

What is a Set?

A set is an unordered collection of **unique** items — duplicate values are automatically removed.

```
numbers = {1, 2, 2, 3, 3, 3}
print(numbers)  # {1, 2, 3}
```

What is a Dictionary?

A dictionary stores data as **key-value pairs**, allowing lookup by a named key instead of a numeric position.

```
person = {"name": "Akash", "role": "Data Engineer"}
print(person["name"])  # Akash
```

Feature	Set	Dictionary
Structure	Unique values only	Key-value pairs
Syntax	{1, 2, 3}	{"key": "value"}
Use case	Removing duplicates	Structured, labeled data

Key takeaway: Sets are useful for de-duplication; dictionaries are useful for structured, labeled data — similar to a real-world lookup table.

5. Conditionals (if / elif / else)

Conditionals allow a program to make decisions and execute different code depending on whether a condition is true or false.

```
age = 20

if age < 18:
    print("Minor")
elif age < 60:
    print("Adult")
else:
    print("Senior")
```

How it works:

- **if** — checks the first condition
- **elif** — checked only if the previous condition was false
- **else** — runs only if none of the above conditions were true

Python evaluates conditions **top to bottom** and executes the first block that evaluates to True; the rest are skipped.

Key takeaway: Conditionals are the foundation of decision-making logic — essential for data validation (e.g., "if a value is null, skip it").

6. Loops (for / while)

Loops are used to repeat a block of code multiple times.

for loop — used when the number of iterations is known or based on a collection.

```
for i in range(5):  
    print(i)      # 0, 1, 2, 3, 4
```

while loop — used when the loop should continue until a condition becomes false.

```
count = 0  
while count < 5:  
    print(count)  
    count += 1
```

Loop Type	When to Use
for	Iterating over a known range or collection (list, string, etc.)
while	Repeating until a condition changes - number of iterations unknown upfront

Key takeaway: Loops are critical in data engineering for processing records one at a time — e.g., looping through rows of a CSV file or API response.

Day 1 Summary — Python Core I

Concept	One-Line Recap
Python	A simple, readable language that acts as the connective layer across the data stack
Variables & Data Types	Named containers holding a value of a specific kind (int, float, str, bool)
Lists & Tuples	Ordered collections - list is changeable, tuple is fixed
Sets & Dictionaries	Set = unique values; Dictionary = labeled key-value data
Conditionals	Decision-making logic based on true/false conditions
Loops	Repeating an action - over a known range (for) or until a condition changes (while)

Overall takeaway: These building blocks — variables, collections, conditionals, and loops — are the foundation of every data pipeline you'll write later: cleaning, filtering, and transforming records all come back to these five concepts.